

Quiz 03 — Dynamic Arrays and Amortized Analysis

Estructuras de Datos · Universidad Nacional de Colombia · [analysis.pdf](#)

Bryan Gustavo Llontop Angulo · Adrien Roho · Diego Fernando Castillo Albino

1. Representación interna

DynamicArray (P1). Guardamos `data` (arreglo genérico, casteado desde `Object[]`), `size` y `capacity`, con `data.length = capacity`. `append` agrega en `data[size]` y hace `size++`; si antes `size == capacity`, primero llama a `resize(capacity*2)`, que crea un arreglo nuevo y copia los `size` elementos uno por uno (sin `Arrays.copyOf` ni `System.arraycopy`, como pide el enunciado). `removeLast` solo hace `size--` y pone `null` en la celda liberada; nunca reduce `capacity`. `get/set` validan que el índice esté en `[0, size)`.

Invariantes de DynamicArray:

1. $0 \leq \text{size} \leq \text{capacity}$ y $\text{capacity} \geq 1$.
2. `data.length = capacity` siempre (el arreglo físico nunca es más chico ni más grande que la capacidad declarada).
3. Las posiciones $0, 1, \dots, \text{size} - 1$ contienen los elementos válidos, en el mismo orden en que se insertaron; el resto del arreglo puede tener basura o `null`, pero ninguna operación pública lo lee.

Snake: arreglo circular (P3). La representación de la Pregunta 2 (`append` + desplazar todo para borrar la cola) es lenta porque la cola siempre vive en la celda física 0. La solución es dejar de exigir que la cola esté fija en la celda 0: usamos un arreglo circular, donde la serpiente puede dar la vuelta al arreglo y aun así cada operación se calcula con una cuenta simple, sin mover a los demás elementos.

a) **Variables de estado.** `snake` (el arreglo físico de `Position`), `size` (cuántas posiciones tiene la serpiente ahora), `capacity` (tamaño del arreglo físico) y `head`, que no es la celda de la cabeza sino la celda *donde se va a escribir la próxima cabeza*. No guardamos la cola como variable aparte porque siempre se puede calcular a partir de `head` y `size` (ver b).

b) **De índice lógico a posición física.** Todo se apoya en una sola cuenta modular, aplicada dos veces. Primero, la celda física de la cola es

$$\text{tail} = (\text{head} - \text{size} + \text{capacity}) \bmod \text{capacity}.$$

Sumamos `capacity` antes de aplicar `%` porque en Java el resto de una resta negativa puede salir negativo, y necesitamos que el resultado quede siempre entre 0 y `capacity-1`. Una vez que tenemos `tail`, la celda física del elemento lógico i (contando desde la cola, con $0 \leq i < \text{size}$) es simplemente

$$\text{físico}(i) = (\text{tail} + i) \bmod \text{capacity}.$$

Con $i = 0$ obtenemos la cola y con $i = \text{size} - 1$ obtenemos la celda justo antes de `head`, que es la cabeza. Como `tail` ya quedó en `[0, capacity)` y a lo sumo le sumamos $\text{size}-1 < \text{capacity}$, el módulo vuelve a dejar el resultado en un índice válido del arreglo.

c) **Operaciones.**

- `addHead(p)`: si `size == capacity`, primero `resize(capacity*2)`. Luego `snake[head] = p`, `head = (head+1) % capacity` y `size++`.
- `removeTail()`: calcula `tail` con la fórmula de arriba, guarda `p = snake[tail]`, pone `snake[tail] = null` y hace `size--`.
- `get(i)`: valida $0 \leq i < \text{size}$ y devuelve `snake[(tail+i) % capacity]`.

Ninguna de las tres recorre ni desplaza los demás elementos: por eso `addHead` y `removeTail` no dependen de `size` (salvo, en `addHead`, cuando toca hacer `resize`).

d) **resize(newCapacity).** Se crea un arreglo nuevo de tamaño `newCapacity` y se copia, para $i = 0, \dots, \text{size}-1$, `newArray[i] = get(i)` (usando todavía el `head/capacity` viejos). Al terminar, `snake = newArray`, `capacity = newCapacity` y `head = size`. Como `get(i)` ya recorre la serpiente de cola a cabeza sin importar en qué celdas físicas estaba cada quien, el `resize` deja los elementos en el mismo orden lógico sin importar cómo estuvieran repartidos antes.

e) **Invariantes de Snake.**

1. $0 \leq \text{size} \leq \text{capacity}$, con $\text{capacity} \geq 1$, y `snake.length = capacity`.
2. $0 \leq \text{head} < \text{capacity}$.
3. Para todo $0 \leq i < \text{size}$, la celda $(\text{tail}+i) \% \text{capacity}$ guarda el elemento i (con $i = 0$ la cola y $i = \text{size} - 1$ la cabeza); las demás celdas son `null` o basura y ninguna operación pública las lee.
4. El orden lógico se conserva siempre: `addHead` agrega en la posición lógica `size` (nueva cabeza), `removeTail` quita la posición lógica 0, y `resize` copia en ese mismo orden.

f) **Traza (N1..N6).** Partiendo de `size=capacity=2`, cola A , cabeza B (`head=0`). En cada movimiento primero se hace `addHead` y luego, si no comió, `removeTail` (tal como en `Main.java`).

Mov.	Arreglo físico completo	head	tail	size	cap	resize	Lógico (cola→cabeza)
Inicial	[A, B]	0	0	2	2	—	A, B
N1 (no come)	[·, B, C, ·]	3	1	2	4	sí (2 → 4, copia 2)	B, C
N2 (come)	[·, B, C, D]	0	1	3	4	no	B, C, D
N3 (no come)	[E, ·, C, D]	1	2	3	4	no	C, D, E
N4 (no come)	[E, F, ·, D]	2	3	3	4	no	D, E, F
N5 (come)	[E, F, G, D]	3	3	4	4	no	D, E, F, G
N6 (no come)	[·, E, F, G, H, ·, ·, ·]	5	1	4	8	sí (4 → 8, copia 4)	E, F, G, H

En N1: como `size==capacity` ($2=2$), `addHead(C)` primero hace `resize(4)` (copia A, B), después escribe C y queda `size=3`; luego `removeTail` borra A. En N3 a N5 la serpiente “da la vuelta”: las celdas físicas 2, 3, 0 guardan, en ese momento, a C, D, E en orden lógico. En N6 vuelve a pasar lo mismo que en N1 pero con capacidad 4.

2. Análisis de complejidad

P1 (c, d). `get`, `set` y `removeLast` solo tocan una celda fija, sin ningún ciclo: $\Theta(1)$ worst-case. `append` es $\Theta(1)$ casi siempre, pero cuando `size == capacity` dispara un `resize` que copia `size` elementos: $\Theta(n)$ en el peor caso. Esto no contradice que el costo amortizado sea constante, porque un `append` caro solo puede pasar cuando el arreglo se acaba de llenar, y justo después la capacidad se duplica: hacen falta otros `capacity` `appends` baratos ($O(1)$ cada uno) antes de que vuelva a haber un `resize`. Sumando todos los `resizes` desde una capacidad inicial C_0 hasta hacer n `appends`, el total copiado es $C_0 + 2C_0 + 4C_0 + \dots < 2n$, así que el trabajo extra de todos los `resizes` juntos es $O(n)$: repartido entre los n `appends` da $O(1)$ amortizado por operación, aunque una sola pueda costar $\Theta(n)$.

P2 (a–d). `REMOVE_TAIL` desplaza los `size–1` elementos restantes una posición a la izquierda: $\Theta(k)$, con k el tamaño de la serpiente. En cada movimiento sin comer, `MOVE` hace un `append` ($O(1)$ amortizado) y un `REMOVE_TAIL` sobre un tamaño transitorio de $k+1$: $\Theta(k)$ por movimiento, y por lo tanto $\Theta(mk)$ para los m movimientos. La afirmación del estudiante es *falsa*: que `append` sea $O(1)$ amortizado es una propiedad de las secuencias de `append`, pero `MOVE` también ejecuta `REMOVE_TAIL`, y sumando el costo real de todos los movimientos, $\sum_{t=1}^m \text{costo}(\text{MOVE}_t) \geq \sum_t \text{costo}(\text{REMOVE_TAIL}_t) = mk$; entonces el costo amortizado de `MOVE` es $\Theta(k)$, no $O(1)$ (para k grande esto no es constante). El **bottleneck** es `REMOVE_TAIL`: como la cola siempre está anclada a la celda física 0, quitarla obliga a recorrer y desplazar toda la serpiente. Se elimina desacoplando índice lógico e índice físico, como en la representación circular de la sección anterior.

P3. Con la representación circular, `get` y `removeTail` tienen costo fijo, con una sola cuenta módulo: $\Theta(1)$ worst-case. `addHead` es $O(1)$ amortizado, y solo en el peor caso (cuando toca `resize`) cuesta $\Theta(\text{size})$. Ningún movimiento normal recorre los k elementos.

3. Demostración del análisis amortizado (P4)

Llamamos P al tamaño *permanente* de la serpiente, es decir su tamaño entre un movimiento y el siguiente. P empieza en 2 y solo aumenta cuando la serpiente come: cada manzana lo sube en 1, y un movimiento sin comer lo deja igual (`addHead` y `removeTail` se cancelan). Entonces, tras g manzanas, $P \leq 2 + g$. El tamaño más grande que la estructura llega a tener en algún instante (incluyendo el pico justo después de `addHead` y antes de `removeTail`) es como mucho $P + 1 \leq g + 3$; llamamos $S = g + 3$ a esa cota.

a) Aggregate Method. Un `resize` solo ocurre dentro de `addHead`, cuando `size == capacity`, y copia exactamente `capacity` elementos. Como la capacidad inicial es 2 y se duplica cada vez, las capacidades en las que hay `resize` son $2, 4, 8, \dots, 2^{j-1}$, donde 2^j es la primera potencia de 2 que alcanza o supera S (es decir $2^{j-1} < S \leq 2^j$). El total de elementos copiados es

$$2 + 4 + \dots + 2^{j-1} = 2^j - 2 \leq 2(g + 2) = 2g + 4 \leq 2g + 2 = O(g).$$

b) Por qué la suma es lineal. Cada `resize` se dispara únicamente cuando `size == capacity`, así que el número de elementos que copia es el mismo que la capacidad que tenía el arreglo en ese momento: el término 2^r de la suma corresponde al `resize` que ocurrió cuando la capacidad era 2^r . Además, como la capacidad se duplica, cada `resize` copia más elementos que *todos* los anteriores juntos ($2^r > 2^0 + 2^1 + \dots + 2^{r-1} = 2^r - 1$), así que la suma completa la domina su último término: si el último `resize` ocurrió con capacidad $2^{k-1} < S$, entonces $\sum_{r=0}^{k-1} 2^r = 2^k - 2 < 2S$. Por eso el costo total de crecer es lineal y no cuadrático en el número de elementos que llegó a tener la estructura.

c) Costo total y amortizado de addHead. Cada movimiento hace un trabajo base de $\Theta(1)$ (escribir una posición y actualizar `head/size`), lo que da $\Theta(m)$ en total para m movimientos. A eso se le suma el costo de las copias por `resize`, que ya vimos que es $O(g)$. El total nos da $\Theta(m) + O(g) = \Theta(m)$, porque $g \leq m$. Repartiendo ese total entre los m `addHead`, el costo amortizado por `addHead` es $\Theta(m)/m = O(1)$.

d) Papel de m y de g . m (el número de movimientos) determina el costo base: cada movimiento, coma o no, hace una escritura que tiene costo constante, así que esa parte del costo total es $\Theta(m)$. g (el número de manzanas)

es lo único que determina cuántas veces se llena el arreglo, y por lo tanto cuánto se copia: si la serpiente nunca come ($g = 0$), nunca hace falta más de un `resize` inicial sin importar cuántos movimientos haga.

e) Potential Method. Definimos la función potencial

$$\Phi = 2 \cdot \text{size} - \text{capacity}.$$

Φ nunca es negativo. Justo después de cualquier `resize`, $\text{size} = C + 1$ (se acaba de insertar el elemento que llenó el arreglo) y $\text{capacity} = 2C$, así que $\Phi = 2(C + 1) - 2C = 2 > 0$. A partir de ahí, el tamaño permanente P solo puede subir, nunca baja hasta el próximo `resize` porque dentro de un mismo movimiento `removeTail` solo deshace lo que ese mismo `addHead` acababa de sumar, así que Φ no puede volver a bajar por debajo de donde dejó el último `resize`. Por lo tanto $\Phi \geq 0$ siempre.

Costo amortizado = costo real $+\Delta\Phi$:

- `addHead` sin `resize`: costo real = 1 (una escritura); $\Delta\Phi = +2$ (`size` sube 1, `capacity` no cambia); amortizado = $1 + 2 = 3$.
- `addHead` con `resize` ($\text{size}=\text{capacity}=C$ antes de insertar): costo real = $C + 1$ (copiar C y escribir 1); Φ pasa de $2C - C = C$ a $2(C + 1) - 2C = 2$, entonces $\Delta\Phi = 2 - C$; amortizado = $(C + 1) + (2 - C) = 3$.
- `removeTail`: costo real = 1; $\Delta\Phi = -2$ (`size` baja 1); amortizado = $1 - 2 = -1$.

En los dos casos de `addHead` el amortizado es el mismo valor constante (3), y `removeTail` incluso da -1. Como en un movimiento sin comer se ejecuta un `addHead` y un `removeTail`, ese movimiento cuesta como mucho $3 + (-1) = 2 = O(1)$; uno que come cuesta $3 = O(1)$. Por qué el potencial alcanza para pagar el próximo `resize`: `removeTail` solo le quita a Φ el crédito que el `addHead` de ese mismo movimiento acababa de depositar, nunca crédito de `resizes` anteriores; por eso $\Phi \geq 0$ se mantiene y, al llegar al siguiente `resize` cuando de nuevo $\text{size}=\text{capacity}=C'$, el potencial disponible es $\Phi = 2C' - C' = C'$, que es exactamente lo que cuesta copiar en ese `resize`.

4. Políticas de crecimiento (P5)

Sea $C_0 = 2$ la capacidad inicial y n la longitud máxima que llega a alcanzar la estructura

a) El momento en que crece no importa. Mientras no haya contracción (la capacidad nunca baja), un `resize` se dispara la primera vez que `size` alcanza la capacidad actual, sin importar en qué número exacto movimientos ocurrió cada aumento de tamaño. Las capacidades por las que se pasa, $C_0, C_1, \dots, C_k$, dependen solo de la política de crecimiento y de n , no del orden en que se llegó ahí; y cada `resize` en capacidad C_r copia exactamente C_r elementos, porque así se dispara siempre. Entonces el total copiado, $\sum_r C_r$, es el mismo sin importar cuándo comió la serpiente. Para que el adversario pudiera aumentar el número de copias necesitaría hacer que la capacidad también baje al vaciarse, y así hacer un patrón que hace crecer un poco, encoger, volver a crecer, repitiendo `resizes` cerca de un mismo umbral en vez de subir una vez hasta n .

b–e) Las tres políticas. Justo después de un `resize` ya se insertó el elemento que lo disparó, así que $\text{size} = C + 1$ con la nueva capacidad C_{new} ; el espacio sin usar es $C_{\text{new}} - (C + 1)$.

Política	# Resizes	Total copiado	Amortizado crecimiento	por Espacio sin usar tras <code>resize</code>
$C + 1$	$n - 2 = \Theta(n)$	$\sum_{C=2}^{n-1} C \approx \frac{n^2}{2} = \Theta(n^2)$	$\Theta(n^2)/n = \Theta(n)$	0 (0%)
$2C$	$\Theta(\log n)$	$< 2n: \Theta(n)$	$< 2: O(1)$	$\approx C$ ($\approx 50\%$)
$\lceil 1.5C \rceil$	$\Theta(\log n)$	$< 3n: \Theta(n)$	$< 3: O(1)$	$\approx C/2$ ($\approx 33\%$)

Con $C+1$ casi cada inserción después de llenarse dispara un `resize`, y cada uno copia casi todo lo que había, de ahí el $\Theta(n^2)$. Con $2C$ las capacidades van $2, 4, 8, \dots$ y con $\lceil 1.5C \rceil$ van $2, 3, 5, 8, 12, 18, \dots$; en ambas cada `resize` copia más que todos los anteriores juntos, así que el total queda acotado por una constante veces n .

f) El trade-off. Con $2C$ y $\lceil 1.5C \rceil$ hay pocos `resizes` y poco trabajo total de copia, pero se reserva más memoria de la que se usa justo después de cada `resize` (50% con $2C$, 33% con $1.5C$); con $C+1$ casi no se desperdicia memoria, pero se paga con muchísimos `resizes` y un costo total cuadrático, es decir ya no hay costo amortizado constante. Subir el factor de crecimiento ($1.5 \rightarrow 2$) da menos `resizes` y menos copias, pero más memoria vacía tras cada `resize`; bajarlo hace lo contrario. No se puede tener a la vez memoria perfectamente ajustada y costo amortizado constante.

Familia $C_{\text{new}} = \lceil \alpha C \rceil$. No hace falta duplicar exactamente: alcanza con cualquier $\alpha > 1$ constante que no dependa de n ni se vaya acercando a 1. Como cada `resize` multiplica la capacidad por al menos α , después de k `resizes` la capacidad es al menos $\alpha^k C_0$, así que para llegar a n hacen falta solo $k \approx \log_\alpha(n/C_0) = \Theta(\log n)$ `resizes`. Igual que antes, cada `resize` copia más que todos los anteriores juntos, así que la suma de las copias queda acotada por una constante veces el último término:

$$\text{total copiado} < \frac{\alpha}{\alpha - 1} n = O(n),$$

y el costo amortizado por crecimiento es $O(1)$ (con $\alpha = 2$ da menos de $2n$; con $\alpha = 1.5$ da menos de $3n$, tal como en la tabla). Lo que hace falta es que $\alpha - 1$ sea una constante positiva fija: si α se acerca a 1 a medida

que n crece (o si usamos $\mathbb{C}+1$, que equivale a $\alpha \rightarrow 1$), esa fracción $\frac{\alpha}{\alpha-1}$ se dispara y el total de copias vuelve a ser $\Theta(n^2)$, perdiendo el amortizado constante. En conclusión: cualquier $\alpha > 1$ fijo funciona igual de bien en cuanto al orden de complejidad; lo único que cambia con α es la constante de copias y cuánta memoria queda sin usar.